



# Understanding Ansible Modules

Ansible modules are the fundamental building blocks of automation, serving as standalone scripts that perform specific tasks whilst abstracting complexity from users. This comprehensive guide explores module architecture, usage, and development best practices for DevOps engineers.

# What Is an Ansible Module?

Ansible modules are reusable, standalone scripts designed to perform specific automation actions. They abstract away underlying complexity, allowing users to execute sophisticated operations through simple, declarative syntax.

Most modules are written in **Python**, leveraging its robust ecosystem and readability. For Windows-specific tasks, modules are typically developed in **PowerShell** to integrate seamlessly with the Windows environment.

Two critical characteristics define well-designed modules:

- **Idempotency:** Modules must produce the same result regardless of how many times they're executed, ensuring safe repeated runs
- **Structured output:** All modules return data in JSON format, enabling consistent parsing and integration across playbooks

Modules can be installed via the library directory or distributed through **Ansible Content Collections**, providing flexible deployment options.



# Discovering Available Modules with **ansible-doc**



## Display Module Information

The **ansible-doc** command-line utility serves as your primary reference tool for module documentation, providing comprehensive details directly from your terminal.



## Access Documentation Strings

View DOCUMENTATION strings containing short descriptions, parameter specifications, and real-world usage examples for any installed module.



## Flexible Module Naming

Query modules using either fully qualified collection names (e.g., `ansible.builtin.copy`) or short names (`copy`) for quick reference.

Using **ansible-doc** enables rapid understanding of module parameters, requirements, and expected behaviour without leaving your development environment, significantly accelerating playbook development workflows.

# Listing Available Modules

## Quick Module Discovery

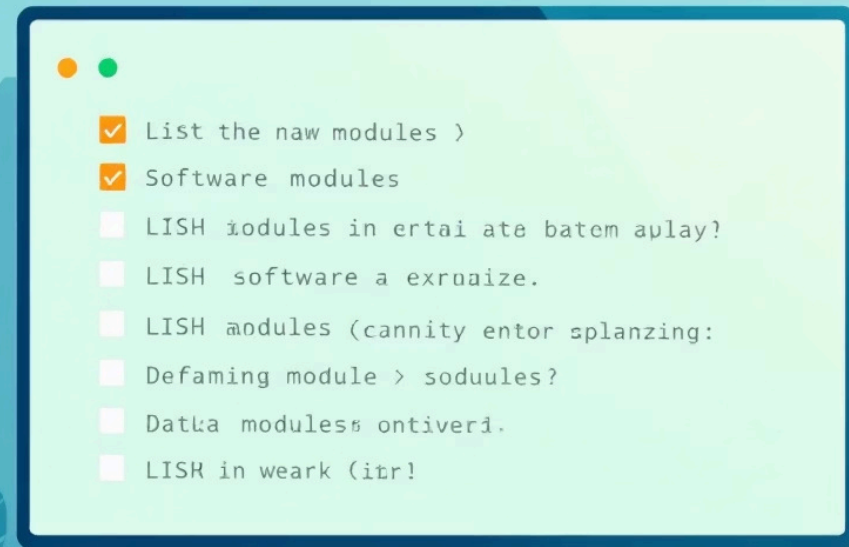
The **ansible-doc -l** command provides a comprehensive list of all available modules in your Ansible environment, complete with brief descriptions for each.

This command is invaluable when exploring available functionality or searching for modules that match specific automation requirements. The output includes modules from core Ansible, installed collections, and custom modules in your library paths.

```
$ ansible-doc -l
```

```
copy Copy files to remote
locations
file Manage files and
properties
template Template a file
out to target
yum Manages packages
with yum
apt Manages apt
packages
```

```
...
```



# Retrieving Detailed Module Documentation

01

## Specify the Module

Run **ansible-doc** followed by the module name to retrieve comprehensive documentation

02

## Review Module Options

Examine available parameters, their types, default values, and whether they're required or optional

03

## Check Requirements

Identify any dependencies, supported platforms, or prerequisite packages needed for the module

04

## Study Examples

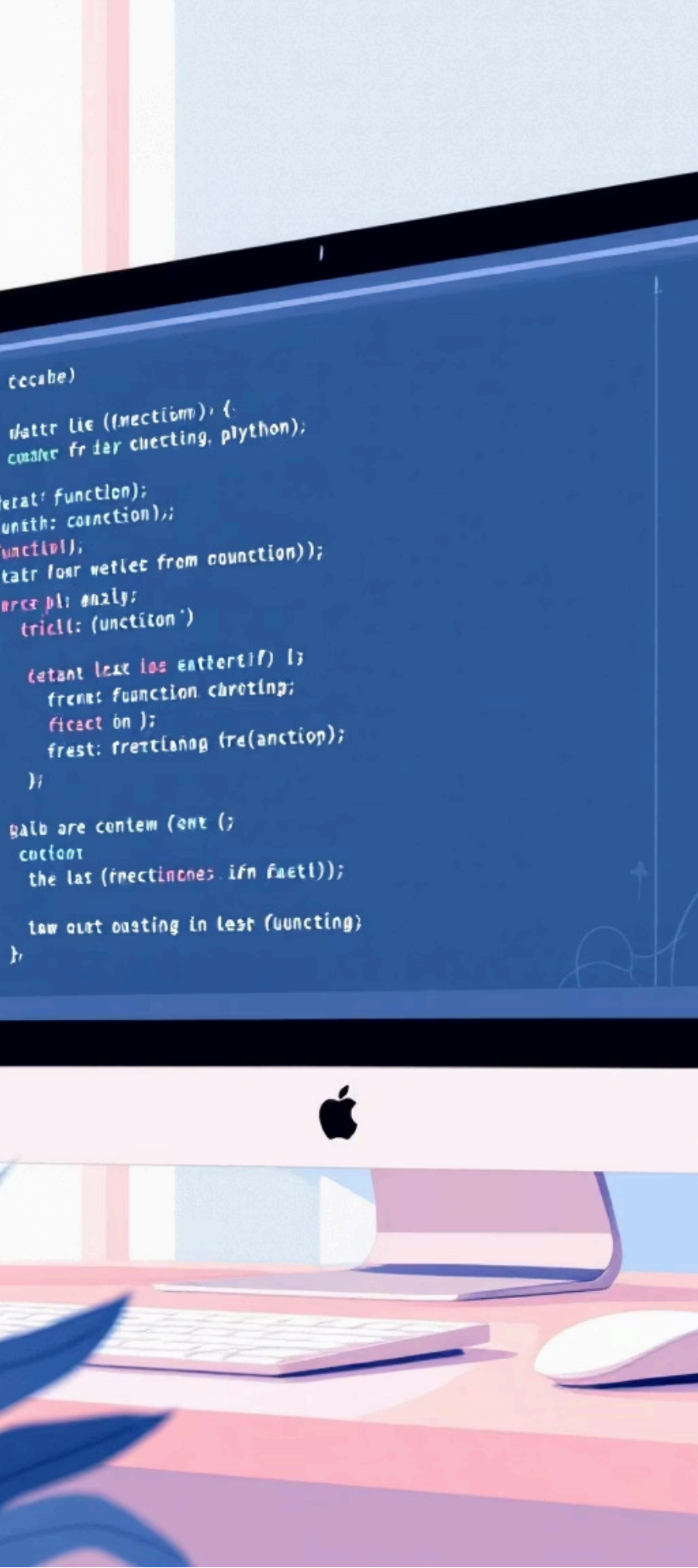
Review practical usage examples demonstrating common patterns and best practices

05

## Understand Return Values

Learn what data the module returns, enabling effective use with register and subsequent tasks

This detailed documentation is essential for understanding proper module usage within playbooks, ensuring you leverage full functionality whilst avoiding common pitfalls.



# Developing Custom Ansible Modules

Custom modules extend Ansible's capabilities to meet specific organisational requirements. Whilst modules can be written in any language, **Python** is the standard choice due to Ansible's `module_utils` library, which provides common functions for argument parsing, JSON output, and error handling.

1

## ANSIBLE\_METADATA

Defines module metadata including status, supported versions, and maintenance information

2

## DOCUMENTATION

Contains structured YAML describing the module's purpose, parameters, requirements, and author details

3

## EXAMPLES

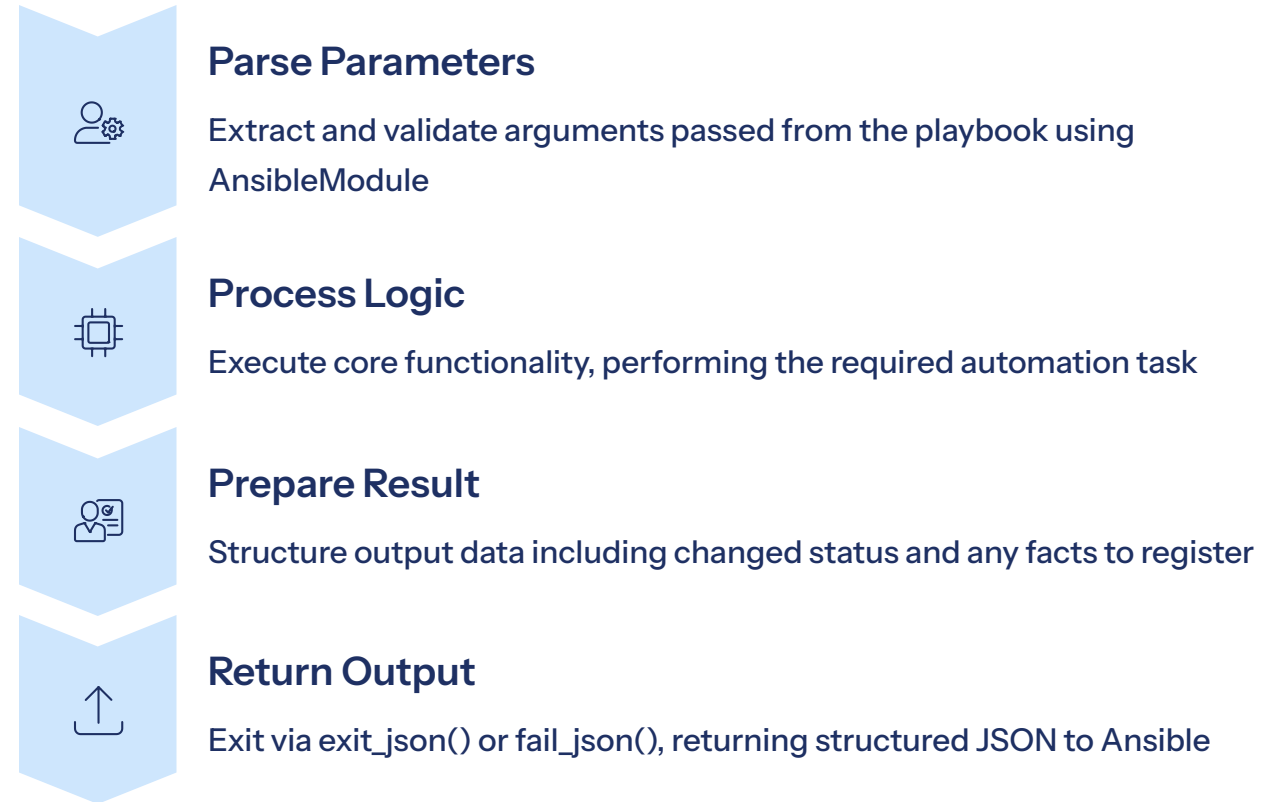
Provides real-world usage examples demonstrating typical invocation patterns

4

## RETURN

Documents the structure and content of data returned by the module upon execution

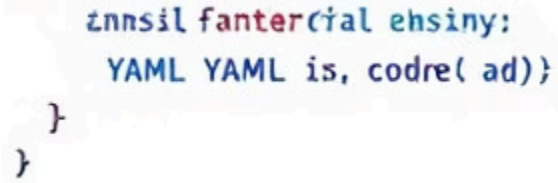
# Module Execution Flow



## Example: hello.py Module

A simple custom module that accepts a 'name' parameter, prints a personalised welcome message, and returns it as an Ansible fact. This demonstrates the fundamental pattern: accept input → process → return JSON output.





## Module Invocation

Use the **register** keyword to capture module output into a variable, then leverage **ansible.builtin.debug** to display results for verification and troubleshooting.

```
- name: Display output
  ansible.builtin.debug:
    var: result
```



# Enabling Custom Modules

1

## ANSIBLE\_LIBRARY Variable

Set the environment variable to point to your custom module directory: **ANSIBLE\_LIBRARY=./library**

2

## User Plugin Directory

Place modules in **~/.ansible/plugins/modules/** for user-specific availability

3

## System Plugin Directory

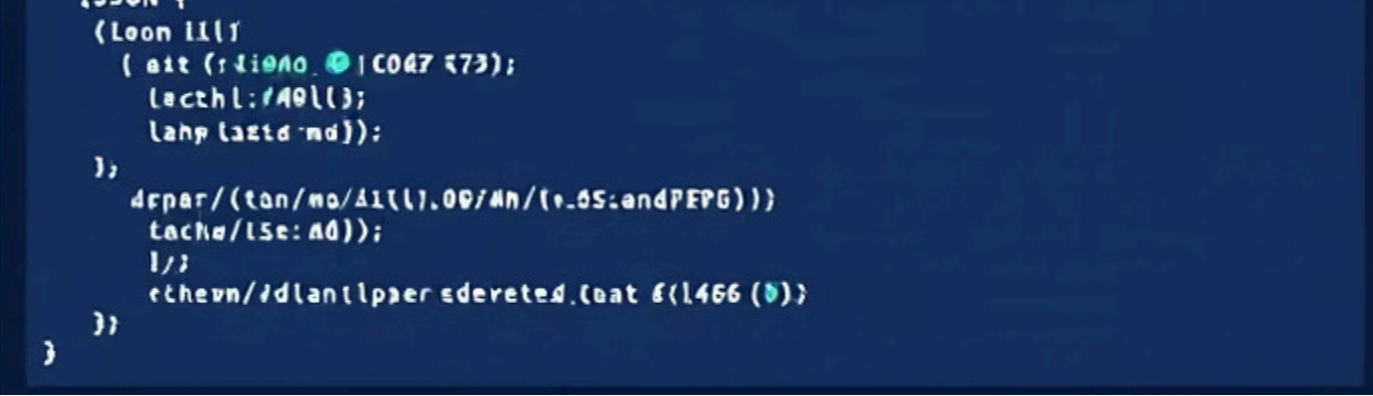
Install modules system-wide in **/usr/share/ansible/plugins/modules/** for organisation-wide access

4

## Ansible Collections

Package modules as part of an Ansible Collection for structured distribution and versioning

Execute your playbook with the library path specified: **ANSIBLE\_LIBRARY=./library ansible-playbook -i inventory playbook.yaml**



# Understanding Module Output

## JSON Response Structure

Modules return structured JSON containing execution results, changed status, and any registered facts. This standardised format enables consistent handling across all playbooks and roles.

The output includes metadata about execution success, whether changes were made to the target system, and custom data specific to your module's functionality.

```
{
  "changed": false,
  "msg": "Hello, DevOps Team!",
  "ansible_facts": {
    "greeting": "Hello, DevOps Team!"
  }
}
```

By understanding module output structures, you can effectively capture and utilise returned data in subsequent tasks, enabling sophisticated automation workflows with proper error handling and conditional logic.